Now, when running the following test case:

```
Set<Post> tuples = new HashSet<>();
tuples.add(entity);
assertTrue(tuples.contains(entity));

doInJPA(entityManager -> {
    entityManager.persist(entity);
    entityManager.flush();
    assertTrue(tuples.contains(entity));
});
```

The final assertion will fail because the entity can no longer be found in the `Set` collection since the new identifier value is associated with a different `Set` bucket than the one where the entity got stored in.

To fix it, the `equals` and `hashCode` methods must be changed as follows:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Post)) return false;
    return id != null && id.equals(((Post) o).getId());
}

@Override
public int hashCode() {
    return 31;
}
```

When the entity identifier is null, equality can only be guaranteed for the same object references. Otherwise, no transient object is equal to any other transient or persisted object. That's why the `equals` method above skips the identifier check if the current object has a null identifier value.

Using a constant `hashCode` value solves the previous bucket-related problem associated with `Set(s)` or `Map(s)` because, this time, only a single bucket is going to be used. Although in general, using a single `Set` bucket is not very efficient for large collection of objects, in this particular use case, this workaround is valid since managed collections should be rather small to be efficient. Otherwise, fetching a `@OneToMany Set` with thousands of entities is orders of magnitude more costly than the one-bucket search penalty.

When using `List(s)`, the constant `hashCode` value is not an issue at all, and, for bidirectional collections, the Hibernate-internal `PersistentList(s)` are more efficient than `PersistentSet(s)`.

The bidirectional `@OneToMany` association generates efficient DML statements because the `@ManyToOne` mapping is in charge of the table relationship. Because it simplifies data access operations as well, the bidirectional `@OneToMany` association is worth considering when the size of the child records is relatively low.

10.3.2 Unidirectional @OneToMany

The unidirectional `@OneToMany` association is very tempting because the mapping is simpler than its bidirectional counterpart. Because there is only one side to take into consideration, there is no need for helper methods and the mapping does not feature a `mappedBy` attribute either.

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```

Unfortunately, in spite its simplicity, the unidirectional `@OneToMany` association is less efficient than the unidirectional `@ManyToOne` mapping or the bidirectional `@OneToMany` association.

Against any intuition, the unidirectional `@OneToMany` association does not map to a one-to-many table relationship. Because there is no `@ManyToOne` side to control this relationship, Hibernate uses a separate junction table to manage the association between a parent row and its child records.

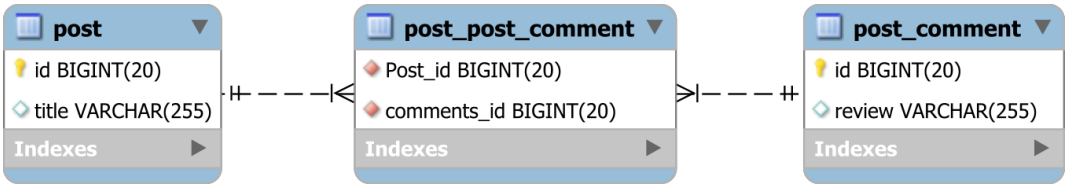

Figure 10.5: The @OneToMany table relationship

The table `post_post_comment` has two foreign key columns, which reference both the parent-side row (the `Post_id` column is a foreign key to the `post` table primary key) and the child-side entity (the `comments_id` references the primary key of the `post_comment` table).

Without going into analyzing the associated data access operations, it is obvious that joining three tables is less efficient than joining just two. Because there are two foreign keys, there need to be two indexes (instead of one), so the index memory footprint increases. However, since this is a regular table mapping for a many-to-many relationship, the extra table and the increased memory footprint are not even the biggest performance issue. The algorithm for managing the collection state is what makes any unidirectional `@OneToMany` association less attractive.

Considering there is a `Post` entity with two `PostComment` child records, obtained by running the following example:

```
Post post = new Post("First post");

post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));

entityManager.persist(post);
```

While for a bidirectional `@OneToMany` association there were three child rows being added, the unidirectional association requires three additional inserts for the junction table records.

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 2)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
```

When removing the first element of the collection:

```
post.getComments().remove(0);
```

Hibernate generates the following DML statements:

```
DELETE FROM post_post_comment WHERE Post_id = 1
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
DELETE FROM post_comment WHERE id = 2
```

First, all junction table rows associated with the parent entity are deleted, and then the remaining in-memory records are added back again. The problem with this approach is that instead of a single junction table remove operation, the database has way more DML statements to execute.

Another problem is related to indexes. If there is an index on each foreign key column (which is the default for many relational databases), the database engine must delete the associated index entries only to add back the remaining ones. The more elements a collection has, the less efficient a remove operation gets.

The unidirectional `@OneToMany` relationship is less efficient both for reading data (three joins are required instead of two), as for adding (two tables must be written instead of one) or removing (entries are removed and added back again) child entries.

10.3.3 Ordered unidirectional @OneToMany

If the collection can store the index of every collection element, the unidirectional `@OneToMany` relationship may benefit for some element removal operations. First, an `@OrderColumn` annotation must be defined along the `@OneToMany` relationship mapping:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostComment> comments = new ArrayList<>();
```

At the database level, the `entry` column is included in the junction table.

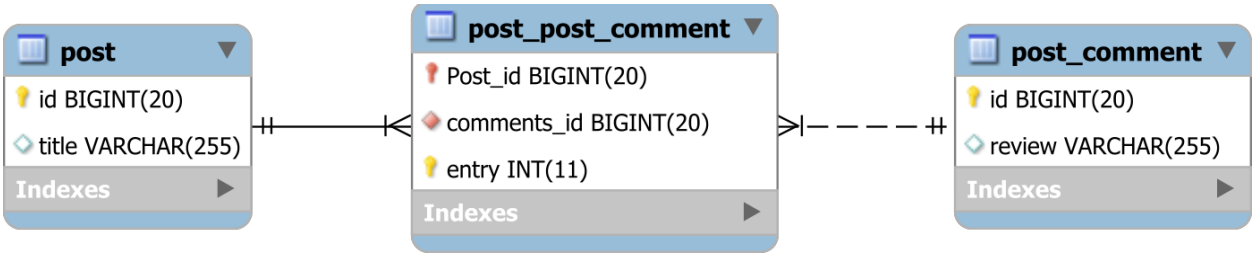

Figure 10.6: The unidirectional @OneToMany with an @OrderColumn

It is better not to mistake the `@OrderColumn` with the `@OrderBy` JPA annotation. While the former allows the JPA provider to materialize the element index into a dedicated database column so that the collection is sorted using an `ORDER BY` clause, the latter does the sorting at runtime based on the ordering criteria provided by the `@OrderBy` annotation.

Considering there are three `PostComment` entities added for a given `Post` parent entity:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

The index of every collection element is going to be stored in the `entry` column of the junction table:

```
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 0, 2)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 1, 3)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 2, 4)
```

When removing elements from the tail of the collection:

```
post.getComments().remove(2);
```

Hibernate only requires a single junction table delete statement:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
DELETE FROM post_comment WHERE id = 4
```

Unfortunately, this optimization does not hold for entries that are located towards the head of the collection. So, if we remove the first element:

```
post.getComments().remove(0);
```

Hibernate deletes the last entry associated with the parent row from the junction table, and then it updates the remaining entries to preserve the same element ordering as the in-memory collection snapshot:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
UPDATE post_post_comment SET comments_id = 4 WHERE Post_id = 1 AND entry = 1
UPDATE post_post_comment SET comments_id = 3 WHERE Post_id = 1 AND entry = 0
DELETE FROM post_comment WHERE id = 2
```